CHAPTER 1 | COMPUTER FUNDAMENTALS, ORGANIZATION & ARCHITECTURE

1.1 The Computer — Characteristics, Functions and Generations

Computer: an electronic machine that accepts data, processes it according to stored instructions, and gives out information. Its five functions are **input, storage, processing, output and control**. The **CPU is called the brain of the computer** because it executes every instruction and controls all other units — memory only stores, and I/O devices only communicate.

Characteristics: the qualities every computer shares:

(1) Speed — works in millionths of a second.

(2) Accuracy — errors come from wrong input, not from the machine.

(3) Diligence — it can work for hours **without creating any error and without grumbling**; it never tires.

(4) Versatility — the same machine does many kinds of work.

(5) Storage — it remembers huge volumes of data.

(6) No I.Q. — it cannot think; it only follows instructions.

Time and Storage Units

Sub-second time units go down in steps of one thousand:

$$milli = 10^{-3},\ micro = 10^{-6},\ nano = 10^{-9},\ pico = 10^{-12},\ femto = 10^{-15},\ atto = 10^{-18}$$

Remember: 1 picosecond = 10^{-12} s and **10^{-15} s = 1 femtosecond** — both asked directly.

Storage units climb in steps of 1024: 8 bits = 1 byte, 4 bits = 1 **nibble** (half byte), 1 KB = 1024 bytes, 1 MB = 1024 KB, 1 GB = 1024 MB. So, **4 MB = 4096 KB**; 2 KB = 4096 nibbles; 3 KB = 24,576 bits; and "4 KB = 4069 bytes" is false — the true figure is **4096** (watch the digit-swap trap). **Cost per bit is highest in registers and lowest in magnetic tape** — cost and speed rise together toward the CPU, capacity runs the other way.

History, Generations and Classification

ENIAC = Electronic Numerical Integrator **and Computer** (the first general-purpose electronic computer); **EDVAC** = Electronic **Discrete Variable** Automatic Computer (early stored-program machine). The generations are identified by their switching technology:

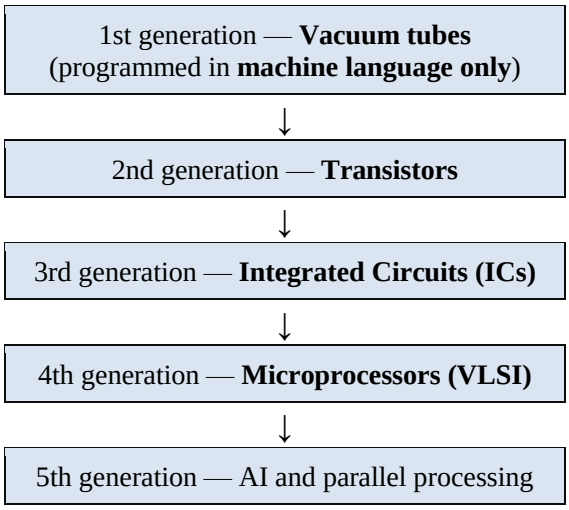


Fig. 1.1 — Computer generations by technology. 'Which generation used transistors?' → the second.

By size and power, computers are classified as **microcomputers, minicomputers, mainframes and supercomputers**. **Mainframes use IBM's 8-bit EBCDIC code**; a **supercomputer's speed is measured in FLOPS — floating-point**

operations per second. In a microcomputer, **memory temporarily holds the data and instructions during processing**, and the **width of the data bus is measured in bits**.

Exam Points — 1.1

- ✦ Diligence = works for hours without error or complaint. Brain of the computer = CPU (asked in nearly every paper).
- ✦ 1 picosecond = 10^{-12} s; 10^{-15} s = 1 femtosecond. 4 MB = 4096 KB; 2 KB = 4096 nibbles; the '4069 bytes' statement is the false one.
- ✦ ENIAC = Electronic Numerical Integrator and Computer; EDVAC = Electronic Discrete Variable Automatic Computer; 2nd generation = transistors; 1st generation programmed only in machine language.
- ✦ Mainframes → EBCDIC; supercomputer speed → FLOPS. Cost of storing a bit is minimum in magnetic tape.

1.2 Number Systems and Conversions

Positional number system: the value of a digit depends on its position (place value). Binary, octal, decimal and hexadecimal are all positional; the **Roman system is non-positional**. In base b the digits run from **0 to (b − 1)** — base 7 has no digit 7, so 7777 is invalid in base 7 and the largest 4-digit base-7 number is 6666; the smallest 4-digit number in **any** base is 1000.

With n bits we can form 2^n distinct patterns, covering the numbers

$$0 \text{ to } 2^n - 1$$

Remember: a binary number of n ones equals $\mathbf{2^n - 1}$: so 111111 = $2^6 - 1$ = **63**. And representing 0 to $2^n - 1$ needs exactly **n bits**.

The Four Systems and How to Convert

Conversion	Method	Worked example from the papers
Decimal → binary/octal/base-r	**Divide by r repeatedly; read remainders bottom-up**	$(325)_{10} = (505)_8$; $(952)_{10} = (1670)_8$; $(213)_{10} = (3111)_4$; $(1715)_{10} = (BAB)_{12}$
Decimal fraction → base r	**Multiply by r repeatedly; read integer carries top-down**	$(10.25)_{10} = (1010.01)_2$; $(25.625)_{10} = (11001.101)_2$; $(34.36)_{10} \approx (100010.010111)_2$; $(929.19)_{10} \approx (1641.141)_8$
Binary → decimal	Sum the powers of 2 where bits are 1	$(11011)_2 = 16+8+2+1 =$ **27**
Hex → decimal	Weight digits by powers of 16 (A=10 … F=15)	$(A5)_{16} = 160+5 =$ **165**
Binary ↔ octal	**Group 3 bits** per octal digit	$(1101010)_2 = 001\|101\|010 =$ **152**; $(562)_8 =$ **101110010**
Binary ↔ hex	**Group 4 bits** per hex digit	$(F4)_{16} =$ **11110100**
Octal ↔ hex	**Always go through binary** (3 bits out, regroup in 4s)	$(726)_8 =$ **1D6**; $(651.124)_8 =$ **$(1A9.2A)_{16}$** — fraction bits group AWAY from the point

Note: For "which value is NOT equal" questions — $(7214)_8$, $(6103)_8$ type — convert once to binary, then read that binary in every base: base 4 takes 2 bits, octal 3 bits, hex 4 bits per digit. For sums across bases like $(89D)_{16} + (259)_{10}$, always bring both to one base first — the answer was **2464** in decimal. Binary division works the same way: $(100001)_2 \div (110)_2 = 33 \div 6 = 5$ (remainder ignored) = **$(101)_2$**.

Exam Points — 1.2

- ✦ Roman = non-positional. Base-b digits: 0 to b−1. Largest 4-digit base-7 number = 6666 (7777 invalid); smallest 4-digit number in any base = 1000 (both tested statements false).
- ✦ 111111 = 63 (2^6−1). Range 0 to 2^n−1 needs n bits.

✦ All the worked conversions in the table above are actual paper questions — practise the octal-fraction-to-hex one, $(651.124)_8 = (1A9.2A)_{16}$, which trips most students.

1.3 Signed Numbers, Complements and IEEE-754

Sign bit: in signed representation the leftmost bit shows the sign: **0 = positive, 1 = negative** (both statements correct).

1's complement: invert every bit.

2's complement: invert every bit, then add 1. This is how computers store negative numbers. The signed range of n bits is:

$$-2^{n-1} \text{ to } +2^{n-1} - 1$$

(1) 2's complement of 1100 = 0011 + 1 = **0100**. Of $(101100)_2$: invert → 010011, add 1 → **010100** (stopping at 010011 gives only the 1's complement — the standard trap).

(2) **−8 in 8 bits**: +8 = 00001000 → invert → 11110111 → +1 → **11111000**.

(3) **−15 needs 5 bits** (4-bit signed range is only −8 to +7): 01111 → invert → 10000 → +1 → **10001**. (The 4-bit pattern 1111 is −1, not −15 — the source key was wrong here and the book's explanation corrects it.)

(4) **−28 in 16 bits** = 1111111111100100 (the option ending …100011 is the 1's complement).

(5) **10000000 in 8 bits = −128** — the MSB carries weight -2^7. It is the one value with no positive counterpart, and therefore **1000 is the only 4-bit number equal to its own 2's complement**.

Remember: negating a 2's-complement number twice returns the original — a quick self-check.

IEEE-754 single precision: a 32-bit float stored as **1 sign bit + 8 exponent bits + 23 mantissa bits**. For $C1C00000_{16}$ = 1100 0001 1100 0000…: sign = 1, and the next eight bits give the exponent field **1000 0011**.

Exam Points — 1.3

✦ 2's complement = invert + 1; range -2^{n-1} to $+2^{n-1}-1$; 4-bit signed range −8 to +7.

✦ 2's complement of 1100 = 0100; of 101100 = 010100; −8 (8-bit) = 11111000; −15 = 10001 (5 bits); −28 (16-bit) = 1111111111100100.

✦ 10000000 (8-bit) = −128; 1000 equals its own 2's complement. IEEE-754 single = 1 + 8 + 23 bits; exponent of C1C00000 = 1000 0011.

1.4 Computer Codes — BCD, Gray, ASCII, Unicode

Code	What it is	The asked facts
BCD (8421)	Each decimal digit coded separately in 4 bits	$(473)_{10}$ = **0100 0111 0011** — 12 bits, while pure binary needs only 9; never confuse the two
Excess-3	Digit stored as (value + 3)	8 = 1000 in 8421 = **1011 in Excess-3** (8+3 = 11); it is a self-complementing code
Gray code	**Only one-bit changes between successive numbers**	Binary → Gray: copy the MSB, then XOR adjacent bits; an n-bit number needs **n − 1 XOR gates** (6-bit 110011 → **5 XOR gates**); $(743)_8$ → binary → Gray = 100010010
ASCII	7-bit American Standard Code for Information Interchange — 128 characters	**A = 65, a = 97** (case differs by 32); EXAM in ASCII-7 = 1000101 1011000 1000001 1001101; ASCII covers only **1, at most 2 languages** — its limitation
Extended ASCII / **EBCDIC**	8-bit codes	EBCDIC = **Extended Binary Coded Decimal Interchange Code**, IBM's mainframe code
ISCII	8-bit Indian extension of ASCII	**Indian Standard Code for Information Interchange** — for Devanagari-based scripts

Unicode / UTF-8	Universal character set	**UTF-8** is the common encoding — **8-bit base unit**, variable length, ASCII-compatible; created precisely because ASCII could not cover world languages

Two small definitions complete the topic. The **collating sequence** is the assigned ordering among the characters a computer uses — it decides how text sorts. A **cipher** is a **character-for-character or bit-for-bit transformation without regard to the linguistic structure** of the message (a code, by contrast, replaces whole words).

Exam Points — 1.4

- ✦ BCD of 473 = 010001110011. 8421's 1000 = Excess-3's 1011. Gray = one-bit-change code; binary→Gray needs n−1 XOR gates.
- ✦ ASCII full form; A = 65; EXAM encoding; ASCII handles 1–2 languages only. EBCDIC and ISCII full forms. UTF-8 = 8-bit Unicode encoding.
- ✦ Collating sequence = character ordering. Cipher = bit-level transformation ignoring linguistic structure.

1.5 Inside the CPU — ALU, Control Unit, Registers

The CPU has **three main parts: the ALU, the Control Unit and the Registers**. RAM is main memory — it sits **outside** the CPU on the system bus, so it is never part of the CPU's internal architecture.

ALU: performs **all arithmetic, logical, comparison, bitwise and shift operations** — all calculations and decision-making happen here. It does **not** do memory addressing (that is the control unit's side). The **XOR** operation outputs 1 **only when the two inputs differ** — the difference detector.

Control Unit: manages and coordinates the entire computer system but performs no actual processing on data (both statements true). It acts as the **nervous system** of the computer. In a **microprogrammed control unit**, the control signals come from **microinstructions and microprograms stored in control memory** — microprogramming means programming the control steps of the computer. The alternative, hardwired control, uses fixed logic (faster, less flexible).

Register	Holds
Program Counter (PC)	**Address of the NEXT instruction** to be executed
Instruction Register (IR)	**The instruction currently being executed**, while the control unit decodes it
Accumulator (AC)	**Data temporarily during arithmetic and logical operations** — one operand in, result back; typically **4 bytes** wide
Stack Pointer (SP)	Address of the **top of the stack** — a **16-bit register in the 8085**; PUSH decrements it, POP increments it
Buffer registers	Data in transit — they **absorb the speed difference** between fast and slow devices
MAR / MDR	Memory address being accessed / data being transferred

The **data path = registers + ALU + buses** — everything the data flows through; the control unit steers it. **CPU registers are the fastest storage** in the machine (then cache, then RAM, then disk). The instruction cycle every instruction passes through:

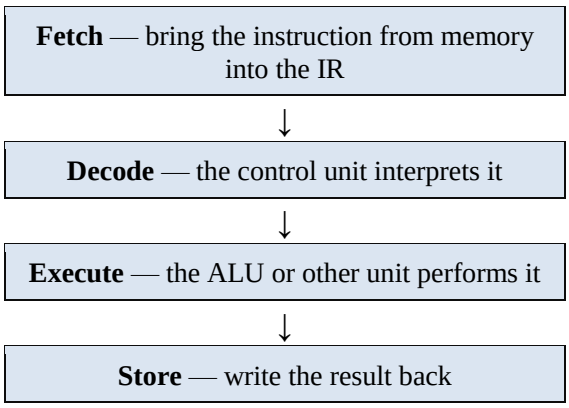


Fig. 1.2 — The instruction cycle. 'Encode' is NOT a phase — the planted wrong option.

Microprocessors

(1) **Microprocessor = the CPU on one chip; microcontroller = a complete computer on one chip** (CPU + memory + I/O).

(2) Generations by word size: 4-bit = 1st, **8-bit = 2nd (Intel 8085)**, 16-bit = 3rd, 32-bit = 4th. The **third generation has a 24-bit address bus → maximum memory 2^{24} = 16 MB**. (8085 is 8-bit; 8086 is 16-bit — don't confuse the pair.)

(3) The microprocessor unit contains **control & timing circuitry, the ALU and registers — all of these**. The **interface unit resolves the difference between the CPU and a peripheral** (buffering + handshaking). The **address bus physically connects** the microprocessor, memory and other parts.

(4) An **array processor** has **a single control unit broadcasting instructions carried out simultaneously by many processing elements** — the SIMD model (a multiprocessor is MIMD). A **software interrupt** is executed **to obtain system services that need privileged instructions** — the program's own deliberate call; an **interrupt** in general is an exceptional condition **caused by an event external to the CPU**.

Note: MSIL (Microsoft Intermediate Language) is the **CPU- and platform-independent instruction set** of .NET, converted to native code by the JIT compiler — a stray fact filed here.

Exam Points — 1.5

- CPU = ALU + CU + registers; RAM is not internal to the CPU. ALU: calculations and decisions, XOR = 1 when inputs differ, no memory addressing. CU: coordinates all, processes nothing, nervous system.
- PC = address of next instruction; IR = current instruction; accumulator holds data during operations; stack pointer marks the stack top (16-bit in 8085).
- Instruction cycle: Fetch → Decode → Execute → Store ('Encode' is fake). Control memory stores microinstructions/microprograms. Data path = registers + ALU + buses. Registers = fastest memory.
- Microcontroller = computer on a chip. 8085 = 8-bit, 2nd generation; 3rd generation max memory = 16 MB. Array processor = SIMD broadcast. Software interrupt = system-service call. Buffer registers absorb speed differences; PCI bus extends the processor bus.

1.6 Instruction Set Architecture and Addressing Modes

ISA (Instruction Set Architecture): the contract between hardware and software — it defines the instructions the CPU can execute. Its key design goal is **balancing simplicity and performance** for easy programming and efficient execution. The recognised types of computer architecture are **micro-architecture, Harvard architecture and the ISA** — "software architecture" is not one. A machine instruction has two parts: **opcode (the operation) + operand (the data or its address)**. Instructions written directly in 1s and 0s are **machine code** — the only language a CPU executes.

RISC	CISC
Reduced Instruction Set Computer	**Complex** Instruction Set Computer
Few, simple, **fixed-length, easily decoded instructions**	Many (100–250) complex, variable-length instructions
Operands come from registers; hardwired control	**Instructions manipulate operands in memory; microprogrammed control**
Most energy-efficient architecture — ARM chips in mobiles	More power-hungry

Addressing Modes

Mode	Where the operand is	Memory references
Immediate	Inside the instruction itself	**0**
Direct (absolute)	**The instruction gives the operand's actual address** — ADD X, Y form	**1**
Indirect	The instruction points to a location that holds the ADDRESS of the operand	**2** — the most of any mode
Register	In a named CPU register	0
Register indirect	**A register holds the operand's memory address** — ADD R1, [R2]; the brackets are the giveaway	1
Implied	Implicit (e.g. accumulator)	0

(1) Only **one bit** in the instruction is needed to distinguish direct from indirect addressing.

(2) A real CPU uses direct, indirect and immediate modes routinely — asked as "the most common addressing technique" with answer **more than one / all of them**.

(3) By operand count: **zero-address = stack machine** (stack addressing IS zero addressing), **one-address = accumulator machine** (ADD X means AC ← AC + X), **three-address** names all operands (ADD R1, A, B).

(4) On a subroutine CALL, **the return address is pushed onto the STACK** (the stack pointer only marks the top; it does not hold the address itself).

Pipelining

A k-stage pipeline needs **k − 1 cycles to fill**; after that **one instruction completes every clock cycle**. The n-th instruction completes at cycle:

$$k + (n - 1)$$

Worked: 5-stage pipeline → after **4** cycles one instruction rolls out per cycle. 7-stage pipeline → the 2nd instruction completes at cycle 7 + 1 = **8**.

Exam Points — 1.6

- ISA goal = balance simplicity and performance; architecture types = micro, Harvard, ISA (not software). Instruction = opcode + operand; machine code = 1s and 0s.
- RISC = fixed-length simple instructions, energy-efficient; CISC = microprogrammed, memory operands, 100–250 instructions.
- Direct = address given (ADD X Y = absolute); indirect = 2 memory references (the most); register indirect = ADD R1,[R2]; 1 bit separates direct/indirect; stack = zero-address; accumulator = one-address; ADD R1,A,B = three-address; CALL pushes the return address on the stack.
- k-stage pipeline fills in k−1 cycles; n-th instruction finishes at k+(n−1): 5-stage → 4; 7-stage 2nd instruction → 8.

1.7 Memory — RAM, ROM and the Hierarchy

Memory is arranged as a hierarchy — fastest and costliest nearest the CPU:

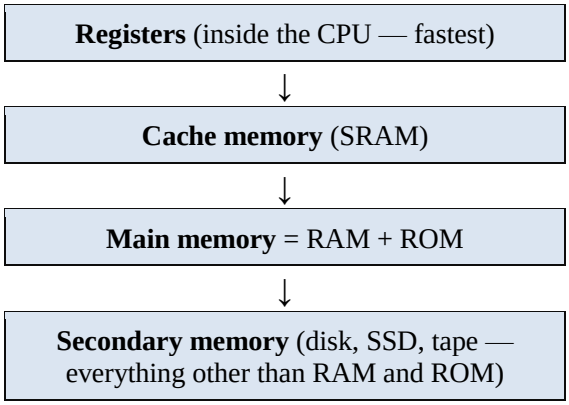
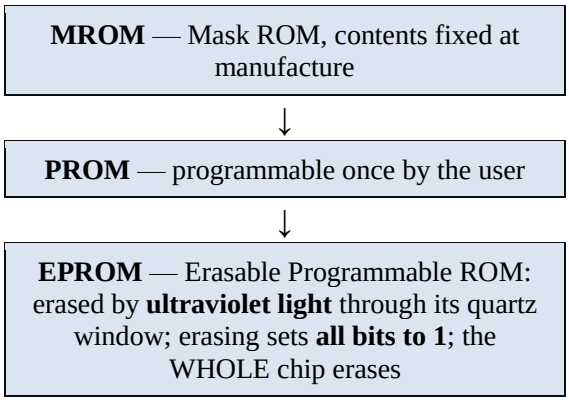


Fig. 1.3 — The memory hierarchy. Speed and cost per bit fall going down; capacity rises. Main memory holding current work vs secondary holding long-term data is a PHYSICAL organisation.

RAM: Random Access Memory — **volatile** (loses data on power-off) working memory with limited capacity; **any location is reached in a fixed time once its address is given** (that constant-time property defines random access; sequential access like tape depends on position). The two kinds of main memory are **ROM and RAM**.

SRAM	DRAM
Stores each bit in a **flip-flop**	Stores each bit as charge on an **electrical capacitor**
No refreshing needed (static)	Charge leaks — **must be refreshed periodically** (dynamic)
Fast, costly — used for cache	Dense, cheap — used for main memory

ROM: non-volatile, read-only — used for **software that does not update**: firmware, the bootstrap loader, the BIOS. Its family, in order of erasability:



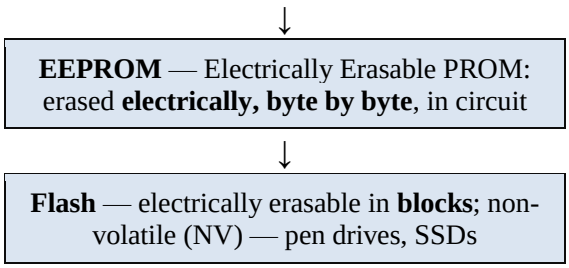


Fig. 1.4 — The ROM family. EAROM (Electrically Alterable ROM) lets a PART of the memory be erased when an error is detected.

Static memory does not change its contents due to external causes (no refresh); dynamic must be refreshed. Related board facts: the **CMOS chip is powered by the motherboard battery** (it keeps BIOS settings and the clock; POST runs at power-ON, the bootstrap program lives in ROM). **Firmware is software permanently stored on a hardware chip** — BIOS in ROM is the classic example. Access mechanisms: **random** (constant time), **sequential** (tape), direct (disk — seek then search), associative (search by content).

Memory Addressing Arithmetic

An address bus of n lines selects 2^n locations, and data lines equal the word width:

$$address\ lines\ =\ log_{\ 2}(number\ of\ words),\quad data\ lines\ =\ word\ size$$

(1) **2K × 16** memory → **11 address lines, 16 data lines**; **64K × 8 → 16 + 8**; **16 KB** needs lines **A0–A13** (2^{14} = 16 K); 10 address lines → **1024** locations.

(2) **FFFF as the last address ⇒ 64 K memory** (2^{16}); a 2-byte unsigned integer spans **0 to 65535**.

(3) Chips needed = (target words ÷ chip words) × (target width ÷ chip width): 128 K × 8 from 16 K × 1 chips = 8 × 8 = **64 chips**.

(4) **Memory cycle time = access time + recovery time** — the minimum delay between two successive memory operations.

Exam Points — 1.7

✦ RAM volatile + limited; secondary memory = storage other than RAM/ROM (both true). Main memory = ROM + RAM. Random access = fixed time at any address.

✦ DRAM = capacitor + refresh; SRAM = flip-flop. ROM stores non-updating software. EPROM = UV erase, erased chip reads all 1s; EEPROM = electrical byte-wise; Flash = NV block erase; EAROM = partial erase; MROM/PROM/EPROM all ROM types.

✦ CMOS powered by motherboard battery. Firmware = software permanent on hardware. 2K×16 → 11+16 lines; 64K×8 → 16+8; 16 KB → A0–A13; FFFF → 64 K; 10 lines → 1024; 128K×8 from 16K×1 → 64 chips; cycle time = access + recovery.

1.8 Cache Memory and Virtual Memory

Cache memory: a small, fast SRAM between the CPU and main memory holding the most-used data — it gives a **speed** illusion. There are exactly **three cache organisations — direct, associative and set-associative** (all three valid):

(1) **Direct mapping**: each main-memory block maps to **only one** possible cache line.

(2) **Fully associative**: a block may go into **any** line.

(3) **Set-associative**: any line within one set — the compromise (direct is 1-way, fully associative is n-way).

A physical address splits left-to-right as **TAG | SET-INDEX | BLOCK-OFFSET**, so:

$$TAG\ bits\ =\ address\ bits\ -\ (set\ bits\ +\ offset\ bits)$$

Worked: 4-way set-associative, 32 KB cache, 1 K block, 32 GB address space: address = 35 bits; offset = 10 (1 K); lines = 32, sets = 32/4 = 8 → 3 set bits; TAG = 35 − 13 = **22 bits**.

Average access time mixes the hit and miss paths by the **hit ratio h**:

$$t_{avg}\ =\ h\ \times\ t_{cache}\ +\ (1\ -\ h)\ \times\ t_{memory}$$

Worked: average 20 ns, cache 10 ns, memory 110 ns → 10h + 110(1−h) = 20 → h = **90%**. Three-level version: cache 5 ns (80%), main 100 ns (99.5%), virtual 10 ms → 0.8×5 + 0.2×(0.995×100 + 0.005×10^7) ≈ **10,024 ns** — one page fault dominates everything.

Virtual memory: the storage-allocation scheme in which **secondary memory can be addressed as though it were part of main memory** — it creates the **illusion of an extremely large main memory** (a size illusion, where cache gives a speed illusion).

Exam Points — 1.8

- ✦ Cache organisations: direct (one line), associative (any line), set-associative (any line in a set) — all three valid.
- ✦ TAG worked example → 22 bits. Hit-ratio worked example → 90%. Three-level average → ≈10,024 ns.
- ✦ Virtual memory = secondary addressed as main = illusion of extremely large main memory.

1.9 Buses, Interrupts, DMA and I/O Organisation

Bus: a **set of parallel lines** carrying bits between components. The system bus has three groups: **address lines** (select the location), **data lines** (carry the value) and **control lines** — which **control the access to and use of the data and address lines**. The number of lines in the data bus is its **width** (in bits). The **PCI bus extends the connectivity of the processor bus**; **SCSI is itself a bus for disks and peripherals, and a serial port can support a printer or scanner** (both statements true). An **interface** connects a peripheral to the bus — it holds the device's data, status and control registers.

Interrupt: an exceptional condition **caused by an event external to the CPU** demanding attention. Valid types: **hardware (external), software, internal (traps)** — there is **no "memory interrupt"**. A **software interrupt is used to simulate an external device from within the program** — the system-call mechanism. **Disabling an interrupt is called masking.** The OS's **I/O subsystem manages input/output operations between processes and I/O devices**.

Data moves between peripherals and memory by exactly three methods — **pipelining is NOT one** (it is a CPU technique):

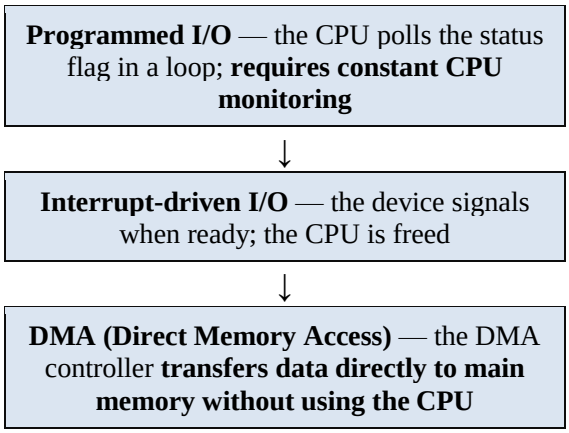


Fig. 1.5 — The three I/O transfer methods, in increasing CPU freedom.

Note: DMA has two modes: **burst transfer** — a whole block of memory words moves **in one continuous burst**; **cycle stealing** — one word at a time between CPU cycles.

Exam Points — 1.9

- ✦ Bus = set of parallel lines; control lines govern data/address lines; data-bus width in bits. PCI extends the processor bus; SCSI is a peripheral bus and serial ports drive printers/scanners.
- ✦ Interrupt types: hardware, software, internal/external — 'memory interrupt' does not exist. Masking = disabling. Software interrupt simulates an external device / requests privileged services.
- ✦ Transfer methods: programmed I/O (constant monitoring), interrupt-driven, DMA (bypasses CPU; burst = continuous block; cycle stealing = word at a time). Pipelining is not an I/O method.

1.10 Input and Output Devices

An input device sends data **into** the computer; an output device brings results **out**. The **input interface converts human form to binary; the output interface transforms information from binary form to human-acceptable form**.

Reader / device	What it reads or does
OMR — Optical Mark Reader	**Presence or absence of pencil/pen marks** — used for **test and survey answer sheets**
OCR — Optical Character Recognizer	Printed characters
MICR	Magnetic-ink characters — **pre-printed on bank cheques**
Barcode reader / Light pen / Digitiser / Joystick	Product codes / on-screen pointing / drawings / game control — all input
Scanner	**Converts printed images into digital format**
Speech recognition	**Speaking data directly into the computer** (what is said); speaker recognition asks who said it
Keyboard	Primary data-entry device; the **Insert key toggles insert and overwrite modes**

Output devices: the **VDU (Visual Display Unit / monitor)** — an output device first, though also a peripheral; **plotter** — draws **high-quality graphics and designs** with pens (CAD/engineering; slower than a laser printer, and output, not input); speakers and headphones; **COM (Computer Output Microfilm)** — an output method, so it **cannot do data entry**. The valid all-output set from the papers: **plotter, VDU, headphone**. An **interactive panel** works as **both input and output** (touch + display). Printer sharpness is measured in **dpi (dots per inch)**.

Impact printers (strike the paper)	Non-impact printers
Dot-matrix — pins strike an inked ribbon (THE impact answer in every paper)	**Ink-jet, laser, thermal**
Line printer — prints a whole line at once; used by **large/mainframe systems**; daisy-wheel	Quieter, faster per page, higher quality

Exam Points — 1.10

- ✦ OMR = marks / answer sheets; OCR = printed text; MICR = cheques; scanner = printed image to digital; speech recognition = spoken input; COM = output only, no data entry.
- ✦ Plotter = output, high-quality graphics. VDU = output device. Interactive panel = input + output. Printer resolution = dpi. Insert key toggles insert/overwrite.
- ✦ Dot-matrix = the impact printer (ink-jet, laser, thermal are non-impact); line printers serve large computer systems.

1.11 Storage Devices, Software Types and Office Basics

(1) **Magnetic tape supports only sequential access** — it is NOT a direct-access device, though it stores large volumes and serves for **backup**. Hard disk, CD and DVD are direct access.

(2) A magnetic disk consists of **cylinders, tracks and sectors**. Capacity order: floppy (1.44 MB) < CD (~700 MB) < magneto-optic < **hard disk** (largest, in TB).

(3) A **buffer** is the storage that **compensates for a difference in the flow of data** between fast and slow units.

System vs application software: system software manages/operates the computer hardware (operating system, **compiler**, drivers, utilities — a set of programs that controls operation and extends processing capability), while **application software helps users perform specific tasks** (word processor, browser, DBMS). **Firmware** is software

permanently stored on a hardware chip. **GIMP is open-source** application software (CorelDRAW, Excel, Illustrator are proprietary). Batch files use **.BAT**. **Virtual reality** systems build computer-generated simulations of real-world activities with interactive graphics. A **debugger** controls execution speed and tracks execution flow; an **editor** searches words and deletes paragraphs.

Note: Matching sets from the papers — MS Office = application, Windows-10 = OS, WinZip = file archiver, AVG = antivirus, NDLI = digital library; UML tools: Power Designer = Sybase, Rational Rose = IBM, Visio = Microsoft, Argo UML = open source. Method: fix the two pairs you are surest of, then eliminate.

Office shortcuts asked: MS-Word: **Ctrl+J = justify**, **Ctrl+T = hanging indent**, **Ctrl+Z = undo**, **Ctrl+H = find & replace** (family: Ctrl+L/E/R = left/centre/right align, Ctrl+Y = redo, Ctrl+K = hyperlink, Ctrl+D = font box). Chrome DevTools = **Ctrl+Shift+I**. **CSV = Comma-Separated-Values** — spreadsheet data as text with a comma/semicolon between cells ("CVS" is the trap spelling). **Google Meet = video conferencing**.

Exam Points — 1.11

- ✦ Tape = sequential only + backup; disk = cylinders, tracks, sectors; buffer evens out data flow; hard disk = maximum capacity.
- ✦ Compiler = system software; GIMP = open source; firmware = permanent software on hardware; .BAT; debugger tracks execution, editor edits text.
- ✦ Ctrl+J justify, Ctrl+T hanging indent, Ctrl+Z undo, Ctrl+H find-replace, Ctrl+Shift+I DevTools; CSV full form; Google Meet = video conferencing.

Quick Revision — TOPPERS FARRE

1. Five functions: input, storage, processing, output, control. CPU = brain. CU = nervous system. Diligence = tireless accuracy.
2. pico = 10^{-12} s; femto = 10^{-15} s. 4 MB = 4096 KB; 1 KB = 1024 B; nibble = 4 bits. Cost/bit: registers highest, tape lowest.
3. ENIAC = Electronic Numerical Integrator and Computer; EDVAC = Electronic Discrete Variable Automatic Computer. Generations: vacuum tube → transistor (2nd) → IC → microprocessor. 1st gen = machine language only.
4. Mainframe → EBCDIC; supercomputer → FLOPS. Data-bus width in bits.
5. Positional systems: binary/octal/decimal/hex; Roman non-positional. Base b digits 0 to b−1; smallest 4-digit number in any base = 1000.
6. n bits → 0 to 2^n-1; all-ones = 2^n-1 (111111 = 63).
7. Decimal→base r: divide, remainders bottom-up; fractions: multiply, carries top-down. Binary↔octal = 3 bits; binary↔hex = 4 bits; octal↔hex through binary.
8. Key conversions: $(325)_{10}=(505)_8$; $(A5)_{16}=165$; $(11011)_2=27$; $(1101010)_2=(152)_8$; $(726)_8=(1D6)_{16}$; $(651.124)_8=(1A9.2A)_{16}$; $(213)_{10}=(3111)_4$.
9. Sign bit: 0 positive, 1 negative. 2's complement = invert + 1; range -2^{n-1} to $2^{n-1}-1$.
10. 2's comp of 1100 = 0100; −8 = 11111000 (8-bit); −15 = 10001 (5 bits needed); 10000000 = −128; 1000 is its own complement.
11. IEEE-754 single: 1 sign + 8 exponent + 23 mantissa.
12. BCD codes each digit in 4 bits (473 → 010001110011). Excess-3 = digit + 3 (8: 1000 → 1011). Gray = one bit changes; binary→Gray needs n−1 XOR gates.
13. ASCII 7-bit, 128 chars, A = 65, covers 1–2 languages; EBCDIC = IBM 8-bit; ISCII = Indian Standard Code for Information Interchange; UTF-8 = 8-bit Unicode encoding.
14. Collating sequence = character ordering. Cipher = bit-for-bit transformation ignoring language.

DSSSB – TGT COMPUTER SCIENCE (PRACTICE KING) BOOK

Call/Message: 7011121290 Available @ Amazon: https://amzn.in/d/07E04o9B

15. CPU = ALU + CU + registers (RAM outside). ALU computes and decides; XOR = difference; CU coordinates but never processes.
16. PC → next instruction's address; IR → current instruction; accumulator → data during ALU work; SP → stack top (16-bit in 8085).
17. Cycle: Fetch, Decode, Execute, Store — no 'Encode'. Control memory holds microinstructions. Data path = registers + ALU + buses. Registers fastest, then cache, RAM, disk.
18. Microcontroller = computer on one chip. 8085 = 8-bit = 2nd generation; 3rd gen max memory 16 MB. Array processor = SIMD. Interface unit matches CPU and peripheral speeds.
19. ISA balances simplicity and performance; architecture types: micro, Harvard, ISA. Instruction = opcode + operand; machine code = binary.
20. RISC: fixed-length, simple, energy-efficient (ARM). CISC: complex, microprogrammed, memory operands.
21. Immediate 0, direct 1, indirect 2 memory references (indirect the most). ADD R1,[R2] = register indirect; ADD X Y = absolute; ADD R1,A,B = three-address; stack = zero-address; accumulator = one-address; CALL pushes return address on the stack; 1 bit separates direct/indirect.
22. Pipeline: k−1 cycles to fill; n-th instruction ends at k+(n−1). 5-stage → 4; 7-stage 2nd → 8.
23. Hierarchy: registers → cache → main (RAM+ROM) → secondary (all storage other than RAM/ROM). Main-vs-secondary split = physical organisation.
24. SRAM = flip-flop, no refresh; DRAM = capacitor, refresh needed. Static memory unaffected by external causes.
25. ROM family: MROM → PROM → EPROM (UV, erased = all 1s) → EEPROM (electrical, byte) → Flash (block, NV). EAROM erases a part. ROM = non-updating software; firmware = BIOS in ROM; CMOS runs on the motherboard battery.
26. Access types: random (constant time), sequential (tape), direct, associative. Memory cycle = access + recovery time.
27. 2K×16 → 11 + 16 lines; 64K×8 → 16 + 8; 16 KB → A0–A13; FFFF → 64 K; 10 lines → 1024; 2-byte unsigned 0–65535; 128K×8 from 16K×1 chips → 64.
28. Cache mappings: direct = one line, associative = any, set-associative = any line in one set. TAG = address − set − offset bits (worked answer 22).
29. $t_{avg} = h \cdot t_{cache} + (1-h) \cdot t_{memory}$; worked h = 90%; three-level worked ≈ 10,024 ns. Virtual memory = secondary addressed as main = illusion of huge memory.
30. Bus = parallel lines; address + data + control (control governs the others); width in bits. PCI extends the processor bus; SCSI = peripheral bus.
31. Interrupts: hardware/software/internal — no memory interrupt; masking = disabling; software interrupt = system call. I/O methods: programmed (polling), interrupt-driven, DMA (burst vs cycle stealing) — pipelining excluded.
32. OMR marks/tests; OCR print; MICR cheques; scanner → digital image; speech recognition = spoken input; COM = output only.
33. Plotter, VDU, headphone = output; interactive panel = both; dpi = printer resolution; Insert toggles overwrite. Dot-matrix and line printers = impact; ink-jet/laser/thermal = non-impact; line printer for mainframes.
34. Tape sequential (backup); disk = cylinders, tracks, sectors; buffer balances flow; hard disk = biggest store.
35. Compiler = system software; GIMP open source; firmware permanent on chip; .BAT batch; Ctrl+J justify, Ctrl+T hanging indent, Ctrl+Z undo, Ctrl+H replace, Ctrl+Shift+I DevTools; CSV = Comma-Separated-Values; Google Meet = video conferencing.

DSSSB – TGT COMPUTER SCIENCE (PRACTICE KING) BOOK

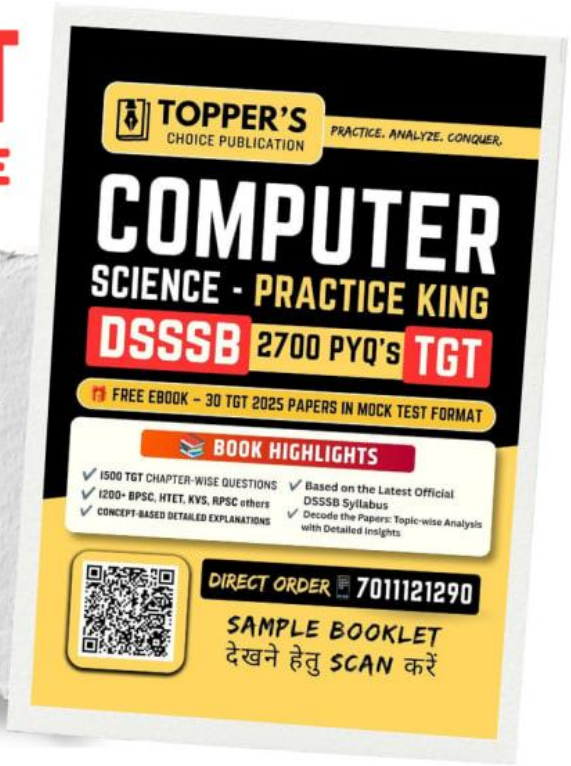
Call/Message: 7011121290 Available @ Amazon: https://amzn.in/d/07E04o9B

7011121290

STUDY THE PATTERN, NOT THE SYLLABUS

DSSSB -TGT
COMPUTER SCIENCE

रटने वाली नहीं,
EXAM को DECODE करने वाली
BOOK!
हर PYQ के साथ
Concept + Trend Analysis



Scan करें और Amazon से
तुरंत ऑर्डर करें

DSSSB – TGT CS (PRACTICE KING) – SUPPORT MATERIAL [TOPPER'S CHOICE]

DSSSB TGT COMPUTER SCIENCE

– COMPLETE PYQ + DEEP ANALYSIS

It includes 2700+ PYQ's (2025, 2024, 2023, 2021 – All Shifts) arranged Subject & Topic Wise, along with Deep Analysis and Exam Tips for quick revision

📖 **BOOK FEATURES**

- ✅ 1,500+ Chapter-wise TGT Practice Questions
- ✅ 1,200+ PYQs from BPSC, HTET, KVS, RPSC & Other Exams
- ✅ Concept-Based Detailed Explanations for Every Important Question
- ✅ Based on the Latest Official DSSSB TGT Computer Science Syllabus
- ✅ Decode the Papers: Topic-wise Trend Analysis & Exam Insights
- ✅ 148 Micro Topics with 100% Syllabus Coverage
- ✅ 15 Previous Papers Analyzed • 1,500 Questions Decoded
- ✅ Frequently Repeated Questions & High-Weightage Topics

🎁 FREE eBook:

📚 1500+ STATIC GK - ONE LINERS

📚 2100+ हिंदी शब्दावली (2025-26)

🎁 **– 30 TGT 2025 Papers in Mock Test Format**

📖 **DEMO SAMPLES**

https://drive.google.com/drive/folders/10YaCwnrGXc_AvJ--57nCvul9AWb0ZfGk

📞 **CALL/MESSAGE:** 7011121290

available at

amazon

📖 https://amzn.in/d/07E04o9B

DSSSB – TGT COMPUTER SCIENCE (PRACTICE KING) BOOK

📞 **Call/Message: 7011121290** 📖 **Available @ Amazon:** https://amzn.in/d/07E04o9B